

Rapport de veille technologique

Contexte

En tant que responsable technique, je dois choisir la technologie backend d'un nouveau projet, avec cinq semaines pour former l'équipe. Trois technologies étaient envisagées : NestJS, Symfony et Spring Boot. Pour les évaluer, j'ai développé une API de kanban board (gestion d'utilisateurs, de listes et de cartes), un projet simple mais complet permettant de juger rapidement les forces et faiblesses de chaque solution.

Présentation des technologies

NestJS est un framework backend Node.js écrit en TypeScript, inspiré de l'architecture d'Angular (modules, contrôleurs, services). Avantages : structure claire et modulaire, typage fort, injection de dépendances, large écosystème JavaScript, courbe d'apprentissage rapide pour qui connaît JS/TS. Inconvénients : performances brutes inférieures aux langages compilés, écosystème plus jeune que ses concurrents, dépendance à Node.js pour les tâches lourdes. Il est particulièrement adapté aux API REST et aux microservices modernes.

Symfony est un framework PHP mature et robuste, très utilisé pour les applications web d'entreprise. Avantages : écosystème riche et stable, excellente documentation, composants réutilisables, forte communauté francophone, idéal pour des applications web complètes. Inconvénients : verbeux, courbe d'apprentissage plus raide, PHP parfois perçu comme moins moderne pour les API pures. Il reste un excellent choix pour des applications web complètes avec rendu de pages côté serveur.

Spring Boot est un framework Java de référence pour les applications d'entreprise. Avantages : extrêmement performant et fiable, adapté aux gros systèmes, écosystème très complet, standard dans les grandes entreprises. Inconvénients : configuration lourde, consommation mémoire élevée, prise en main longue.

Comparaison selon les critères techniques

Critère	NestJS	Symfony	Spring Boot
Runtime / RAM	Léger, démarrage rapide, faible empreinte mémoire	Intermédiaire	RAM élevée au démarrage, très performant en charge
Scalabilité	Architecture non bloquante, scaling horizontal facile	Workers PHP-FPM	Modèle multi-thread
Portabilité	Multiplateforme (Node.js), conteneurs	Multiplateforme (PHP)	Multiplateforme (JVM)
Maturité	Récent (2017), adoption rapide	Mature (2005)	Mature (2003)
Maintenabilité	TypeScript typé, architecture modulaire	Conventions strictes	Typage robuste mais verbeux
Design patterns	Modules, décorateurs, DTO, DI	Services, Event Dispatcher, DI	IOC, AOP, DI

Justification du choix : NestJS plutôt que Symfony et Spring Boot

J'ai choisi NestJS en le comparant à ses concurrents. Face à Spring Boot (Java), NestJS est bien plus léger et rapide à mettre en place, Spring Boot est puissant, mais sa configuration est lourde, sa consommation mémoire élevée et sa prise en main longue, ce qui est disproportionné pour une API à développer en cinq semaines avec une équipe à former.

Face à Symfony (PHP), NestJS repose sur TypeScript, offrant un typage fort et une détection des erreurs dès l'écriture. Symfony excelle pour des applications web classiques rendant leurs pages, mais NestJS est plus naturel pour une API REST pure destinée à un frontend séparé ou une application mobile.

Enfin, sur le plan personnel, j'utilise quotidiennement PHP dans mon alternance, ce projet était l'occasion idéale de sortir de cet environnement et d'ajouter une nouvelle corde à mon arc en montant en compétence sur une technologie moderne. Son architecture modulaire et son écosystème (JWT, Prisma, Swagger) ont permis de construire une API structurée et évolutive efficacement. NestJS offrait donc le meilleur compromis entre montée en compétence, modernité et clarté.

Conclusion

Après comparaison sur les critères de runtime, scalabilité, portabilité, maturité, maintenabilité et design patterns, NestJS s'est imposé comme le choix le plus adapté à ce projet : moderne, structuré, rapide à prendre en main et parfaitement taillé pour une API REST, tout en gardant un code maintenable et évolutif. Enfin, sa faible empreinte mémoire en fait un choix sobre en ressources, cohérent avec une démarche d'éco-conception, tandis que son écosystème facilite une API sécurisée avec authentification JWT, hachage des mots de passe avec bcrypt et validation stricte des données via des DTO.